II-4-6 Multithreaded Sorting Application

Хэрэглэгчийн болон ашиглалтын гарын авлага

F.CSM302 Үйлдлийн систем, Бие даалт 1

2026

Эх сурвалж: Silberschatz, Galvin, Gagne. *Operating System Concepts*, 10th ed., Chapter 4, Programming Projects: “Multithreaded Sorting Application”.

Энэ гарын авлага хоёр хэсэгтэй:

- **А хэсэг. Хэрэглэгчийн заавар:** програмыг ашиглах хүнд зориулсан. Ролт оруулах, ажиллуулах, animation удирдах, өнгө ба төлөвийг унших, түгээмэл алдааг засахыг тайлбарлана.
- **Б хэсэг. Ашиглалтын (operational) заавар:** төслийг шинэ машин дээр татаж, build хийж, тест ажиллуулж, үр дүнг шалгах хүнд (багш, шалгагч, багийн гишүүн) зориулсан.

Агуулга

А хэсэг. Хэрэглэгчийн заавар **3**

А1. Програмын зорилго **3**

А2. Шаардлагатай орчин **3**

А3. Суулгах **3**

А4. Ажиллуулах ба зогсоох **5**

А5. Ролт ба тохиргоо **6**

А6. Animation удирдах **7**

А7. Өнгө, төлөв, тайлбар **8**

А8. Жишээнүүд **9**

А9. Хязгаарлалт **15**

А10. Түгээмэл алдаа ба шийдэл **15**

Б хэсэг. Ашиглалтын (operational) заавар **17**

Б1. Repository татах **17**

Б2. Туршсан орчин **17**

Б3. Build ба ажиллуулах **17**

Б4. Тестийн өгөгдөл **18**

Б5. Автомат тест **18**

Б6. Захын ба алдааны тохиолдлыг гараар шалгах **19**

Б7. Event trace (JSON) **20**

Б8. Мэдэгдэж буй асуудал **21**

Б9. Файлын бүтэц **21**

А хэсэг. Хэрэглэгчийн заавар

А1. Програмын зорилго

Програм бүхэл тоон жагсаалтыг **гурван бодит Java thread** ашиглан эрэмбэлнэ:

1. **Parent thread** жагсаалтыг хоёр хэсэгт хуваана: зүүн хэсэг `[0, mid)`, баруун хэсэг `[mid, n)`, энд $mid = \lceil n/2 \rceil$.
2. Хоёр sorting thread (**Sorter-L**, **Sorter-R**) зэрэг эхэлж, тус бүр өөрийн хэсгийг сонгосон алгоритмаар эрэмбэлнэ. Thread бүр эхлэх, төгсөх индексээ параметрээр авна.
3. Parent хоёр sorter-ийг `join()` хийсний дараа **Merger** thread-ийг эхлүүлнэ. Merger эрэмбэлэгдсэн хоёр хэсгийг **ижил хэмжээтэй хоёр дахь global массив** `result[]` руу нэгтгэнэ.
4. Parent Merger-ийг `join()` хийгээд үр дүнг хэвлэнэ.

Програмыг хоёр янзаар ашиглана:

- **Командын мөр (CLI):** үр дүн, thread бүрийн статистик, хугацааг terminal дээр хэвлэнэ.
- **Web UI (Sorting animation):** хөтөч дээр массивыг багана хэлбэрээр харуулж, thread бүрийн алхмыг өнгөөр ялгана. Энэ animation нь зохиомол биш. Бодит thread-ийн хийсэн үйлдэл бүрийг event болгон бүртгэж, түүнийгээ харуулдаг.

А2. Шаардлагатай орчин

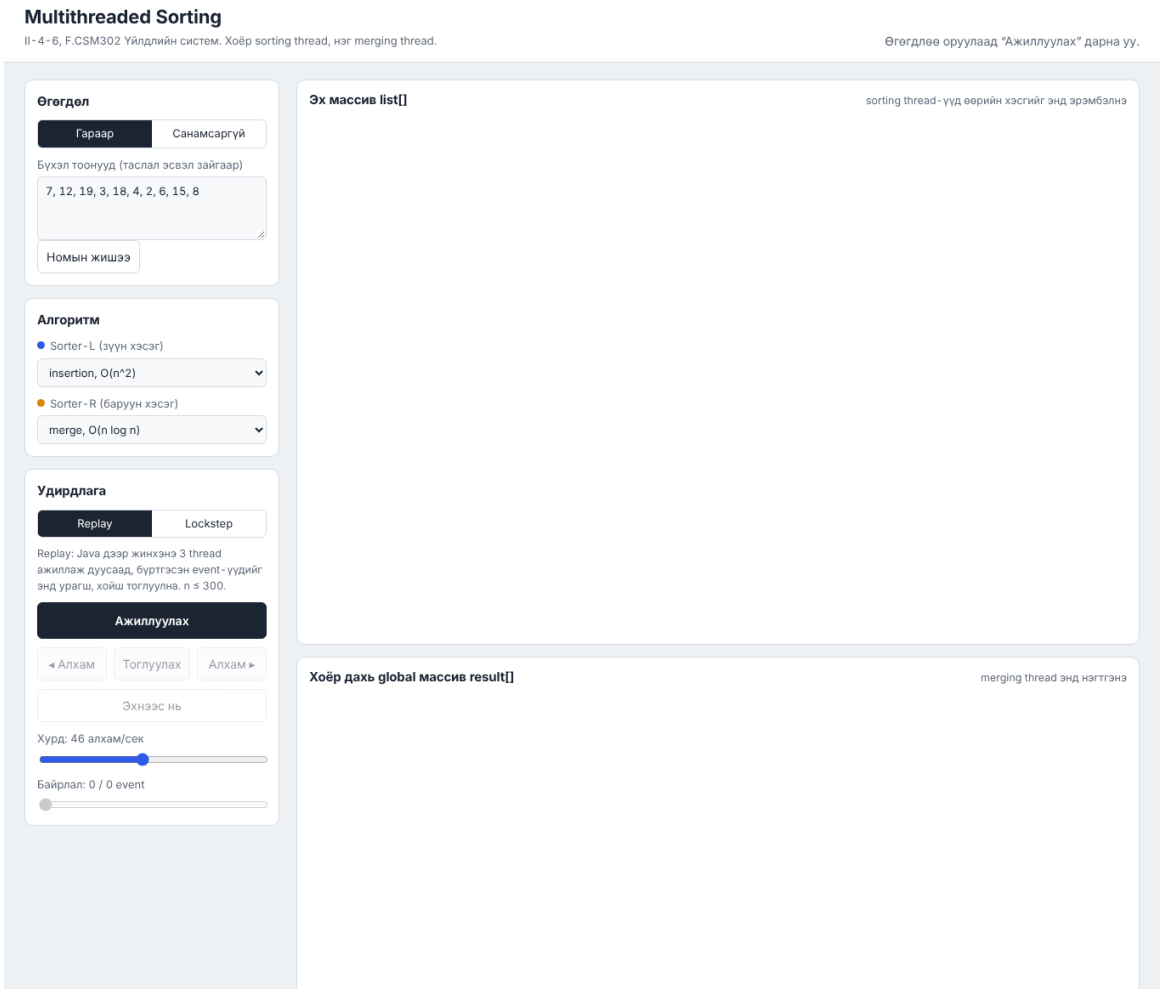
Зүйл	Шаардлага
Java	**JDK 17** буюу түүнээс дээш (`javac` хэрэгтэй, зөвхөн JRE хангалтгүй)
OS	macOS, Linux, Windows (Windows-ийн скриптийг бодит машин дээр туршаагүй, Б8-ыг үзнэ үү)
Хөтөч	Web UI-д: Chrome, Edge, Firefox, Safari-ийн сүүлийн хувилбар
Гадны сан	Хэрэггүй (Maven, Gradle, JUnit, npm ашиглаагүй)
Сүлжээ	Хэрэггүй. Web сервер зөвхөн `127.0.0.1` дээр сонсоно

А3. Суулгах

macOS (Homebrew):

```
brew install --cask temurin@17
export JAVA_HOME=$(/usr/libexec/java_home -v 17)
javac -version
```

Linux (Debian/Ubuntu):



Зураг 1: Web UI-ийн эхлэл

```
sudo apt install openjdk-17-jdk
javac -version
```

Windows: `https://adoptium.net` хаягаас Temurin 17 (JDK)-г суулгаж, суулгах үед “Set JAVA_HOME” болон “Add to PATH” сонголтыг идэвхжүүлнэ. Дараа нь шинэ Command Prompt нээгээд `javac -version` гэж шалгана.

`javac -version` нь `javac 17.x` (эсвэл түүнээс дээш) гэж харуулах ёстой. Дараа нь төслийн хавтсанд `./build.sh` (Windows: `build.bat`) ажиллуулна. `Build OK (javac 17) -> out/` гэж гарвал бэлэн.

A4. Ажиллуулах ба зогсоох

Командын мөр

```
./run.sh --input "7,12,19,3,18,4,2,6,15,8"
```

Гаралт:

```
Оролт: гараас оруулсан
Эх жагсаалт (n=10): [7, 12, 19, 3, 18, 4, 2, 6, 15, 8]
Хуваалт: зүүн [0,5) хэмжээ 5, баруун [5,10) хэмжээ 5
[Sorter-L] tid=18 муж=[0,5) merge: compare=7 swap=0 write=12, 1.347-1.391 ms
[Sorter-R] tid=19 муж=[5,10) merge: compare=5 swap=0 write=12, 1.538-1.549 ms
[Merger] tid=20 нэгтгэл list[0,5) + list[5,10) -> result[0,10): compare=8 write=10,
↪  2.483-2.514 ms
Parent thread бүх thread-ийг join хийлээ.
Эрэмбэлсэн үр дүн: [2, 3, 4, 6, 7, 8, 12, 15, 18, 19]
Шалгалт (Arrays.sort-той харьцуулав): ЗӨВ
Хугацаа: эрэмбэлэх үе 1.574 ms, нийт 2.550 ms
```

- `tid`: Java thread-ийн дугаар. Гурван thread гурван өөр дугаартай байгаа нь тэд тусдаа thread гэдгийг харуулна.
- `муж`: thread-д параметрээр дамжуулсан индексийн муж.
- `compare / swap / write`: тухайн thread-ийн хийсэн үйлдлийн тоо.
- `1.347-1.391 ms`: run эхэлснээс хойш тухайн thread эхэлсэн ба дууссан хугацаа.
- `Шалгалт`: үр дүнг `java.util.Arrays.sort`-той харьцуулсан дүн.

CLI ажиллаад дуусахдаа өөрөө зогсоно. Exit code: `0` амжилттай, `1` үр дүн буруу (хэвийн үед гарахгүй), `2` оролтын алдаа.

Web UI (Sorting animation)

```
./run.sh --web
```

Terminal дээр `Sorting animation сервер ажиллаж байна: http://localhost:8080/` гэж гарна. Хөтөч дээр `http://localhost:8080` хаягийг нээнэ.

- Өөр порт: `./run.sh --web --port 8100`.
- **Зогсоох:** серверийг ажиллуулсан terminal дээр `Ctrl+C`.
- Хөтчийн цонхыг хаахад ажиллаж буй Lockstep session автоматаар цуцлагдана.

A5. Оролт ба тохиргоо

Оролтын дүрэм

- Зөвхөн **бүхэл тоо** (int муж: $-2\,147\,483\,648 \ldots 2\,147\,483\,647$).
- Тусгаарлагч: таслал, цэг таслал, зай эсвэл шинэ мөр. Давхар таслалыг (`1,,2`) хоосон зай гэж үзээд алгасна.
- Сөрөг тоо (`-5`) болон + тэмдэг (`+3`) зөвшөөрнө. Давхардсан утга зөвшөөрнө.
- Файлд `#` тэмдгээс хойших бичвэр тайлбар гэж тооцогдоно.
- **Сондгой n** зөвшөөрнө. Зүүн хэсэг $\lceil n/2 \rceil$, баруун хэсэг $\lfloor n/2 \rfloor$ элементтэй болно (жишээ нь n = 11 үед 6 ба 5).

CLI сонголтууд

Сонголт	Утга
`--input "a,b,c"`	Тоонуудыг шууд оруулах
`a b c ...`	Тоонуудыг аргумент болгон оруулах
`--file PATH`	Текст файлаас унших
`--random N`	N ширхэг санамсаргүй тоо үүсгэх
`--seed S`	Санамсаргүй тооны seed (ижил seed бол ижил жагсаалт)
`--min A --max B`	Санамсаргүй утгын муж (анхдагч 1..100)
`--pattern P`	`random`, `sorted`, `reversed`, `duplicates`
`--algo NAME`	Хоёр sorter-т ижил алгоритм (анхдагч `merge`)
`--algo-left NAME`	Sorter-L-ийн алгоритм
`--algo-right NAME`	Sorter-R-ийн алгоритм
`--list-algos`	Боломжит алгоритмын жагсаалт
`--force`	$O(n^2)$ алгоритмын $n \le 20\,000$ хязгаарыг үл тоох
`--full`	$n > 50$ үед ч массивыг бүтнээр хэвлэх
`--trace FILE`	Бүх алхмыг JSON event trace болгон хадгалах (Б7)
`--web [--port P]`	Web UI-г эхлүүлэх (анхдагч порт 8080)
`-h, --help`	Тусламж

Оролтын эх үүсвэрээс (`--input`, тоонууд, `--file`, `--random`) **яг нэгийг** өгнө.

Алгоритмууд

`Нэр`	Алгоритм	Хүндрэл
`insertion`	Insertion sort	O(n²)
`bubble`	Bubble sort (эрт зогсолттой)	O(n²)
`selection`	Selection sort	O(n²)
`merge`	Merge sort (top-down, туслах буфертэй)	O(n log n)
`quick`	Quick sort (Hoare partition, дунд pivot)	O(n log n) дундаж

Зүүн ба баруун sorter-т **өөр өөр** алгоритм сонгож болно.

Web UI-ийн тохиргоо

Хэсэг	Талбар	Тайлбар
Өгөгдөл	Гараар / Санамсаргүй	Өгөгдлийн эх үүсвэр
	Бүхэл тоонууд	Гараар оруулах жагсаалт. “Номын жишээ” товч номын жишээг бөглөнө
	n, Seed, Min, Max, Хэлбэр	Санамсаргүй жагсаалт үүсгэх тохиргоо
Алгоритм	Sorter-L, Sorter-R	Хэсэг тус бүрийн алгоритм
Удирдлага	Replay / Lockstep	Ажиллах горим (А6)

А6. Animation удирдах

Хоёр горим

	Replay	**Lockstep**
Юу болдог	Java дээр 3 бодит thread бүрэн ажиллаж дуусна. Бүртгэсэн event-үүдийг хөтөч дээр тоглуулна	Java thread-үүд **амьд** хэвээр байна. Үйлдэл бүрийн өмнө `Semaphore`-ийн permit хүлээж, бодитоор `WAITING` төлөвт зогсоно
“Алхам ▸”	Дараагийн бүртгэсэн event-ийг харуулна	Серверт нэг permit өгнө, нэг thread яг **нэг** үйлдэл хийнэ
Хойш алхах	Болно (◂ Алхам, Байрлал гулсуур)	**Болохгүй**, бодит thread-ийг буцааж болдоггүй
Хугацаа	Бодит хурдыг харуулна	Таны товч дарах хоорондын хугацааг багтаана
Хамгийн их n	300	100

Хурд хэмжих бол **Replay**, thread-ийн төлөвийг (`WAITING`, `RUNNABLE`, `TERMINATED`) бодитоор харах бол **Lockstep** горимыг ашиглана.

Товч ба удирдлага

Удирдлага	Үйлдэл	Гарын товч
Ажиллуулах	Java дээр шинэ run эхлүүлнэ	
Тоглуулах / Түр зогсоох	Тасралтгүй тоглуулах эсвэл зогсоох	Space
Алхам ▸	Нэг алхам урагшлах	→
◂ Алхам	Нэг алхам хойшлох (зөвхөн Replay)	←
Эхнээс нь	Replay: эхлэл рүү буцна. Lockstep: шинэ session эхлүүлнэ	
Хурд	1-ээс 5000 алхам/сек (логарифм шкал)	
Байрлал	Гулсуураар дурын event рүү үсрэх (зөвхөн Replay)	

Горимыг солиход өмнөх дүрслэл цэвэрлэгдэж, ажиллаж байсан Lockstep session цуцлагдана. Дараа нь “Ажиллуулах”-ыг дахин дарна.

A7. Өнгө, төлөв, тайлбар

Баганы өнгө

Өнгө	Утга
Цэнхэр	Sorter-L-ийн хэсэг `[0, mid)`
Улбар шар	Sorter-R-ийн хэсэг `[mid, n)`
Ногоон	Merger-ийн `result[]`-д бичсэн утга
Ягаан улаан	Яг одоо харьцуулж буй элементүүд
Хар	Яг одоо сольсон эсвэл бичсэн элемент
Саарал	Merger аль хэдийн авсан элемент
Босоо зураас	Хуваалтын цэг `mid`

Merger ажиллах үед `list[]` дээр `i` (зүүн хэсгийн заагч), `j` (баруун хэсгийн заагч) тэмдэг гарна.

Дэлгэцийн хэсгүүд

1. **Эх массив `list[]`:** хоёр sorter өөрийн хэсгийг энд эрэмбэлнэ.
2. **Хоёр дахь global массив `result[]`:** Merger энд нэгтгэнэ.
3. **Thread-ийн хугацааны шугам:** thread бүр хэзээ ажилласныг бодит `System.nanoTime` хугацаагаар харуулна. Parent-ийн мөрөнд `start()` / `join()` агшнууд босоо зураасаар гарна.
4. **Thread-ийн картууд:** нэр, `tid`, төлөв, муж, үйлдлийн тоо.
5. **Сүүлийн event-үүд:** сүүлд болсон алхмууд, thread бүр өөрийн өнгөтэй.
6. **Үр дүн:** оролт, эрэмбэлсэн үр дүн, шалгалт, хугацаа (trace-тэй ба trace-гүй), event-ийн тоо.

Thread-ийн төлөв

Төлөв (карт)	Утга
Үүсээгүй	Thread объект хараахан үүсээгүй
start() дуудагдсан	Parent `start()` дуудсан, thread эхэлж амжаагүй
Ажиллаж байна	Thread өөрийн ажлыг хийж байна
join() дээр хүлээж байна	(Parent) хүүхэд thread-ээ дуусахыг хүлээж байна
Дууссан	Thread-ийн `run()` дууссан

Lockstep горимд карт дээр нэмэлт **Java төлөв** (`Thread.getState()`) гарна:

`Java төлөв`	Утга
`NEW`	`new Thread()` хийгдсэн, `start()` дуудаагүй
`RUNNABLE`	Ажиллаж байна
`WAITING`	Sorter/Merger: Semaphore permit хүлээж байна. Parent: `join()` дээр хүлээж байна
`TERMINATED`	Дууссан

A8. Жишээнүүд

Жишээ 1. Номын жишээ (Web UI, Replay)

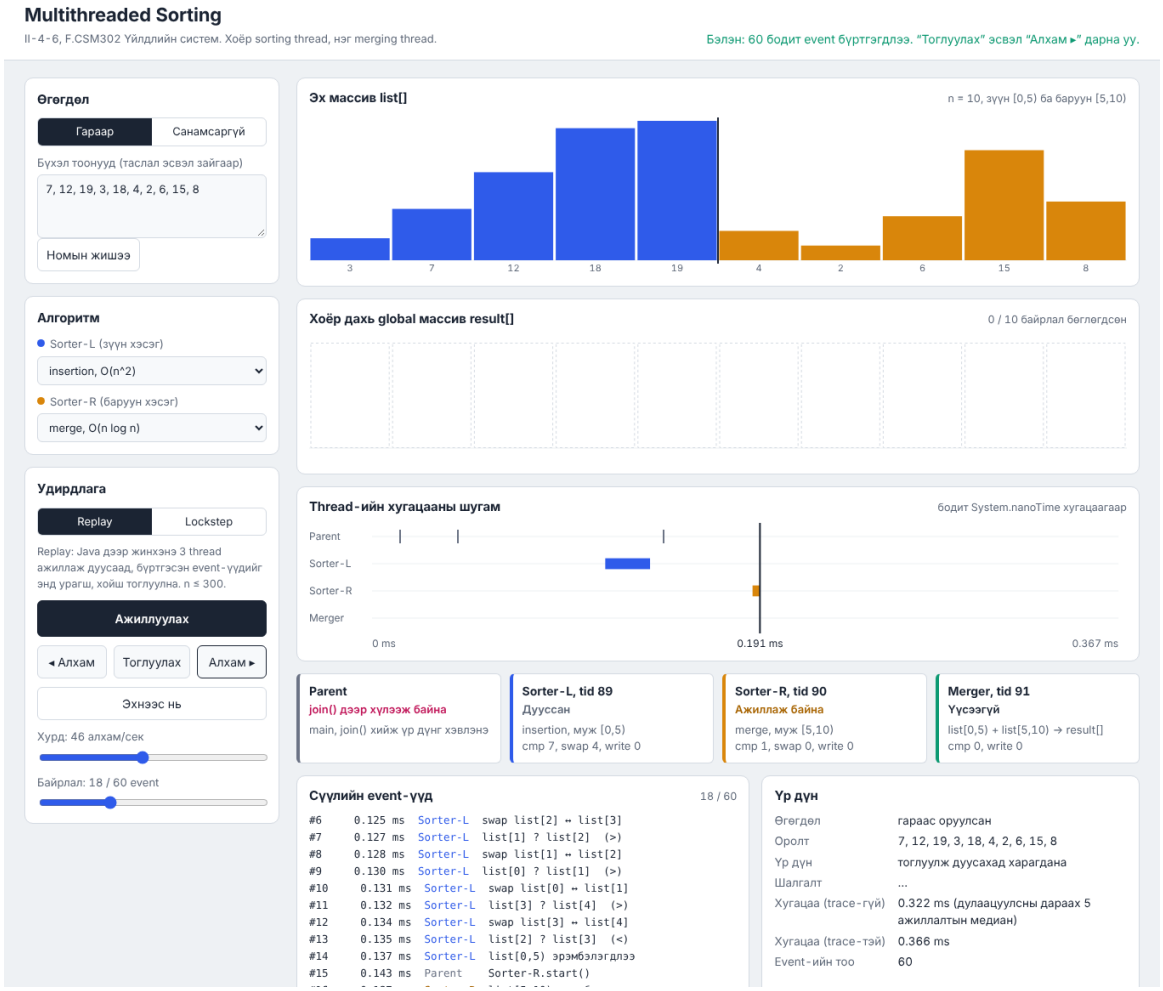
1. `./run.sh --web` ажиллуулж, `http://localhost:8080` хаягийг нээнэ.
2. “Номын жишээ” товчийг дарна: `7, 12, 19, 3, 18, 4, 2, 6, 15, 8`.
3. Sorter-L-д `insertion`, Sorter-R-д `merge` сонгоно.
4. **Ажиллуулах**, дараа нь **Алхам ▸**-ийг хэд дарж, Sorter-L өөрийн хэсгийг insertion sort-оор эрэмбэлж буйг харна (Зураг 2).
5. **Байрлал** гулсуурыг төгсгөл рүү чирвэл Merger `result[]`-ийг ногооноор бөглөж буй нь харагдана (Зураг 3).
6. Төгсгөлд нь `result[] = 2 3 4 6 7 8 12 15 18 19`, Шалгалт: “Эрэмбэлэгдсэн, зөв” (Зураг 4).

Жишээ 2. Сондгой n ба өөр алгоритм (CLI)

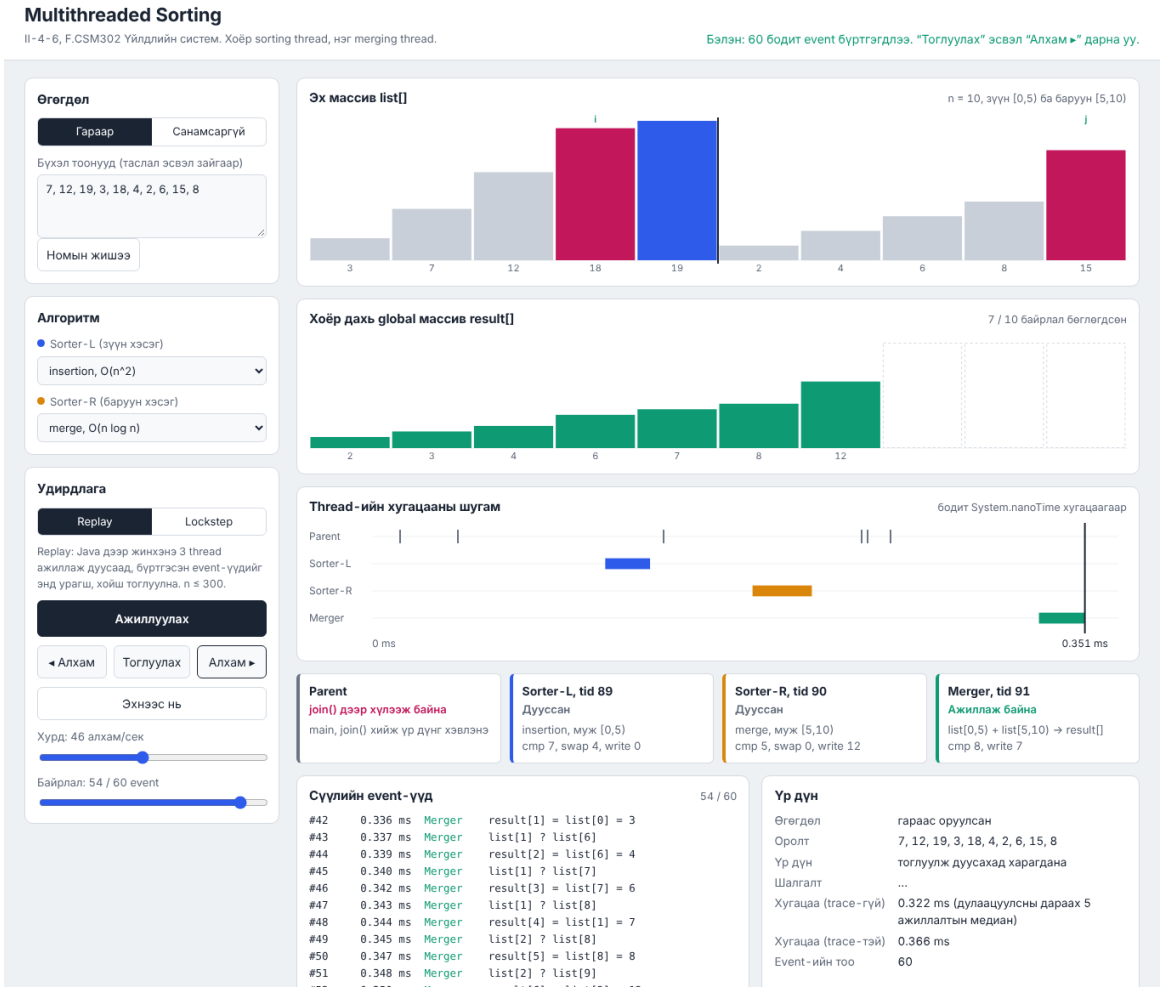
```
./run.sh --file examples/odd-11.txt --algo-left bubble --algo-right quick
```

Хүлээгдэх үр дүн:

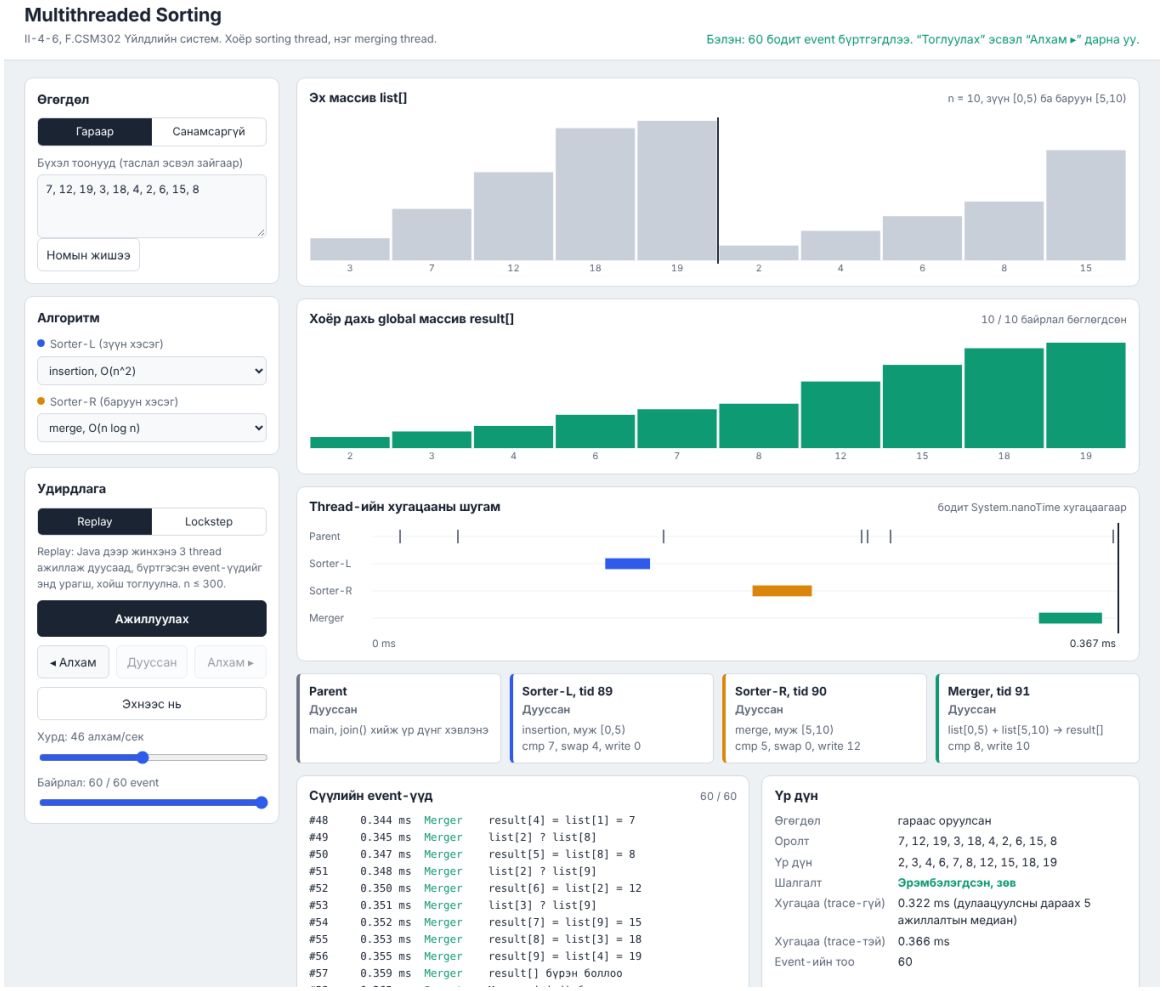
```
Тэмдэглэл: n = 11 сондгой тул зүүн хэсэг 6, баруун хэсэг 5 элементтэй (⌈n/2⌉ / ⌊n/2⌋).
Хуваалт: зүүн [0,6) хэмжээ 6, баруун [6,11) хэмжээ 5
Эрэмбэлсэн үр дүн: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
Шалгалт (Arrays.sort-той харьцуулав): ЗӨВ
```



Зураг 2: Sorter-L ажиллаж байна



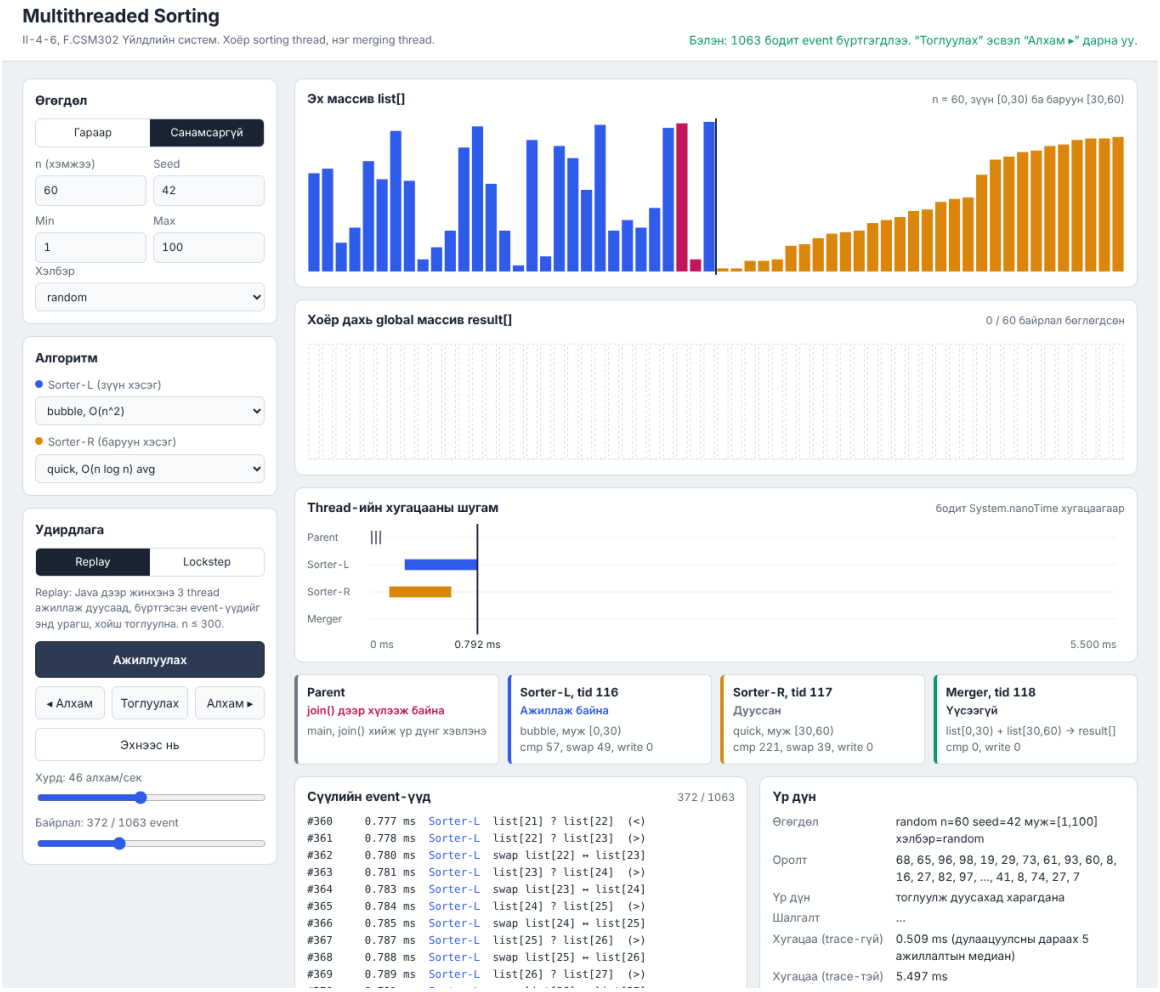
Зураг 3: Merger нэгтгэж байна



Зураг 4: Дууссан төлөв

Жишээ 3. Санамсаргүй 60 тоо (Web UI)

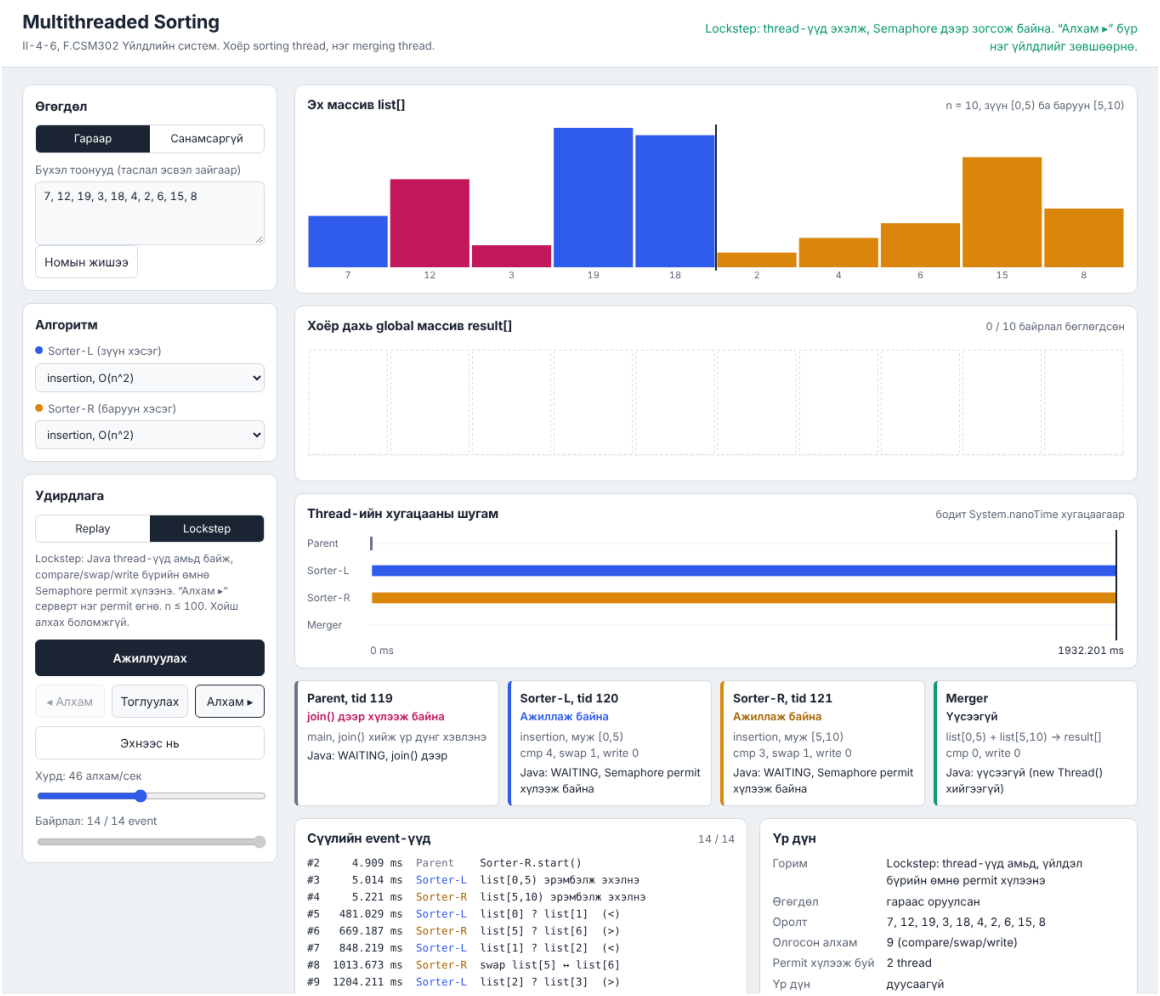
“Санамсаргүй” сонгож n = 60, Seed = 42, Sorter-L = `bubble`, Sorter-R = `quick` гэж тохируулаад Ажиллуулна. Хоёр sorter-ийн хугацааны шугам давхцаж байгаа нь тэд **зэрэг** ажилласныг харуулна.



Зураг 5: Санамсаргүй 60 тоо, bubble + quick

Жишээ 4. Lockstep горим

1. Удирдлага хэсэгт **Lockstep** сонгоно.
2. **Ажиллуулах** дарна. Карт дээр Sorter-L ба Sorter-R: `Java: WAITING, Semaphore permit хүлээж байна`, Parent: `Java: WAITING, join() дээр` гэж гарна.
3. **Алхам ▸** бүрд нэг thread нэг үйлдэл хийнэ. Fair (FIFO) semaphore ашигладаг тул хоёр sorter ээлжилнэ (L, R, L, R … эсвэл R, L, R, L …).
4. **Тоглуулах** дарвал хурдны тохиргооны дагуу permit-ийг тасралтгүй өгнө. Төгсгөлд бүх thread `TERMINATED` болно.



Зураг 6: Lockstep горим: бодит thread-үүд Semaphore дээр WAITING

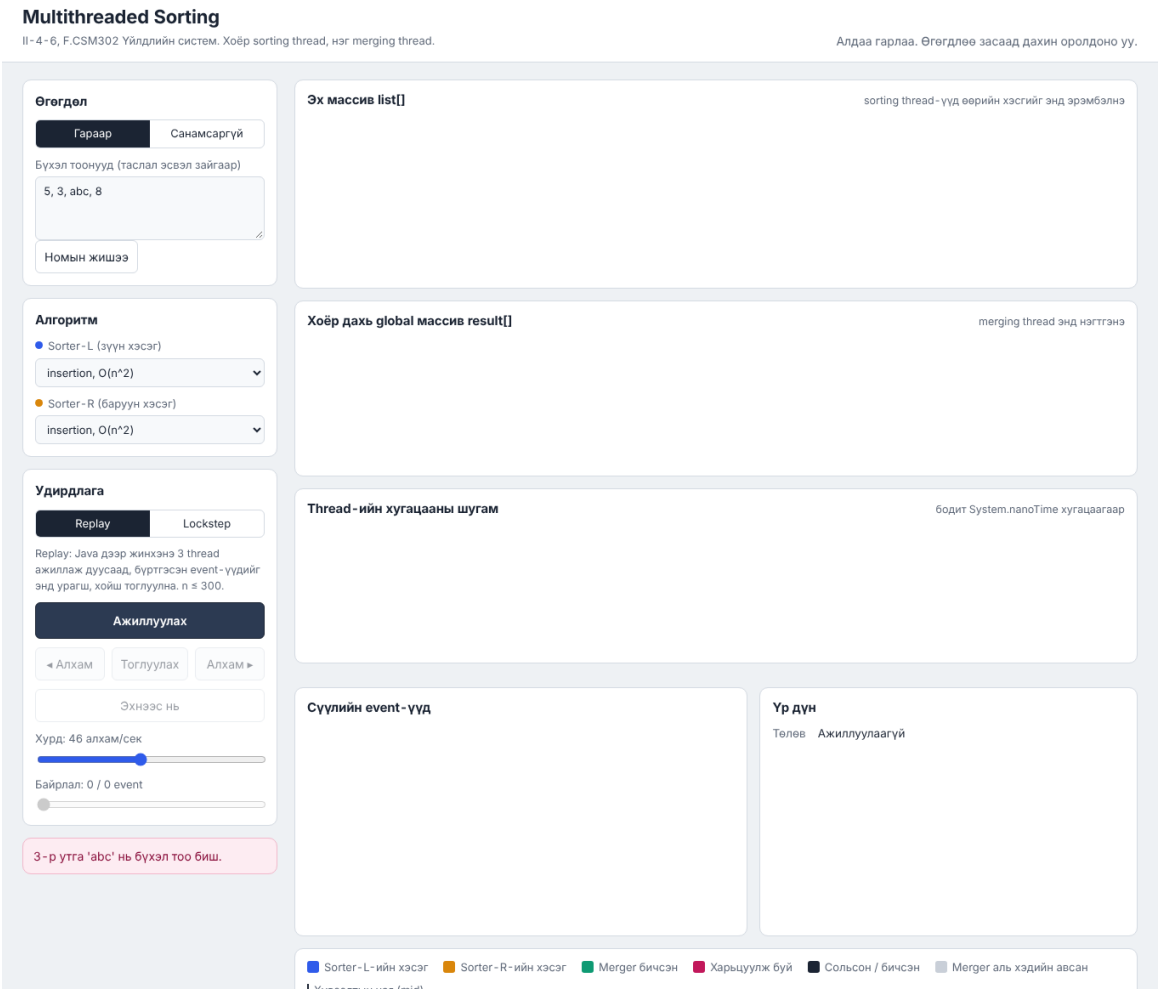
A9. Хязгаарлалт

Зүйл	Хязгаар	Шалтгаан
Утга	int муж	Java `int`
O(n²) алгоритм (insertion, bubble, selection)	n ≤ 20 000 (`--force`-оор давж болно)	Удаан ажиллахаас сэргийлнэ
O(n log n) алгоритм (merge, quick)	n ≤ 1 000 000	Санах ой, хугацаа
`--trace`, O(n²) алгоритмтай	n ≤ 1 000	Алхам бүр нэг event тул файл хэт томорно
`--trace`, merge/quick	n ≤ 10 000	Мөн адил
Web UI, Replay	n ≤ 300	Animation уншигдахуйц байх
Web UI, Lockstep	n ≤ 100	Алхам бүр серверт хүсэлт явуулна
Зэрэг Lockstep session	4	10 минут хөдөлгөөнгүй session автоматаар цуцлагдана

A10. Түгээмэл алдаа ба шийдэл

Алдааны мессеж	Шалтгаан	Шийдэл
`javac олдсонгүй`	JDK суугаагүй	A3-ыг үзнэ үү
`JDK 17+ хэрэгтэй, гэтэл одоогийн javac хувилбар: 11`	Хуучин Java PATH-д байна	macOS: `export JAVA_HOME=$(/usr/libexec/java_home -v 17)`
`out/ олдсонгүй. Эхлээд ./build.sh ажиллуулна уу.`	Build хийгээгүй	`./build.sh`
`Оролтын өгөгдөл өгөөгүй байна`	Оролтын эх үүсвэр алга	`--input`, `--file` эсвэл `--random`
`Оролт хоосон байна`	Хоосон жагсаалт	Дор хаяж нэг тоо
`3-р утга 'abc' нь бүхэл тоо биш.`	Тоо биш утга	Зөвхөн бүхэл тоо
`2-р утга '2.5' нь бутархай тоо.`	Бутархай тоо	Бүхэл тоо оруулна
`2-р утга '3000000000' нь int мужаас хэтэрсэн.`	Хэт том/бага тоо	int мужид оруулна
`insertion нь O(n^2) алгоритм тул n ≤ 20000 ...`	O(n²)-д n хэт их	`merge`/`quick`, n багасгах, эсвэл `--force`
`'foo' гэсэн алгоритм байхгүй`	Алгоритмын нэр буруу	`--list-algos`
`Файл олдсонгүй: ...`	Файлын зам буруу	Төслийн хавтсаас эхлэх зам

Алдааны мессеж	Шалтгаан	Шийдэл
`min (9) нь max (1)-аас их байж болохгүй.`	Муж буруу	min ≤ max
`8080 порт аль хэдийн ашиглагдаж байна`	Портыг өөр програм эзэлсэн	`--port 8100` эсвэл өмнөх серверийг `Ctrl+C`
Хөтөч: `Сервертэй холбогдож чадсангүй`	Сервер зогссон	`./run.sh --web` дахин
`Lockstep session олдсонгүй`	10 минут хөдөлгөөнгүй эсвэл сервер дахин асаасан	“Ажиллуулах” дахин
zsh: `command not found: #`	Командыг тайлбартай нь хуулсан	Зөвхөн командын мөрийг хуулна



Зураг 7: Буруу оролтын алдааны мессеж

Б хэсэг. Ашиглалтын (operational) заавар

Б1. Repository татах

```
git clone https://github.com/Munkhgerelt/F.CSM302-TeamProject-2-4-6.git
cd F.CSM302-TeamProject-2-4-6
```

Б2. Туршсан орчин

Орчин	OS	JDK	Үр дүн
Багийн гишүүний компьютер	macOS 27 (Apple Silicon, aarch64), 10 цөм	Eclipse Temurin 17.0.21+9	Build OK, тест 100/100
Хөгжүүлэлтийн Linux контейнер	Ubuntu 24.04 (amd64), 2 цөм	OpenJDK 21 (`--release 17`)	Build OK, тест 100/100
Windows	Туршаагүй		Б8

`build.sh` нь `javac --release 17` ашигладаг. Тиймээс JDK 21 дээр build хийсэн ч Java 17-д ажиллах код гарна.

Б3. Build ба ажиллуулах

Үйлдэл	macOS / Linux	Windows
Build	`./build.sh`	`build.bat`
CLI	`./run.sh --file examples/book-example.txt`	`run.bat --file examples\book-example.txt`
Web UI	`./run.sh --web`	`run.bat --web`
Тест	`./test.sh`	`test.bat`

Скрипт ажиллуулах эрхгүй бол `chmod +x build.sh run.sh test.sh` хийнэ. `build.sh` нь:

1. `javac`-ийн хувилбарыг шалгана (17-оос доош бол алдаа өгч, зааварчилгаа хэвлэнэ);
2. `src/main/java` доторх бүх `.java` файлыг `out/` руу хөрвүүлнэ;
3. `src/main/resources/web` (HTML/CSS/JS)-ийг `out/` руу хуулна.

`out/`, `out-test/` хавтас, `.class` файлууд нь `.gitignore`-д орсон тул commit хийгдэхгүй.

Б4. Тестийн өгөгдөл

Файл (examples/)	Агуулга	Хүлээгдэх үр дүн
book-example.txt	Номын жишээ, n = 10	2 3 4 6 7 8 12 15 18 19
odd-11.txt	Сондгой n = 11	Хуваалт 6 / 5, 1 2 … 11
duplicates.txt	Давхардсан утга, n = 12	1 1 1 2 2 3 3 3 3 5 5 5
reversed.txt	Урвуу эрэмбэ, n = 10	1 2 … 10
int-limits.txt	Сөрөг тоо, INT_MIN, INT_MAX	-2147483648 -100 -1 0 0 7 100 2147483647
single.txt	Ганц элемент	42, баруун хэсэг хоосон
invalid-letters.txt	abc агуулсан	Алдаа, exit code 2
invalid-overflow.txt	3000000000	Алдаа, exit code 2
empty.txt	Зөвхөн тайлбар	Алдаа, exit code 2

Санамсаргүй өгөгдлийг `--random N --seed S --pattern P` сонголтоор давтагдахуйц үүсгэнэ.

Б5. Автомат тест

```
./test.sh
```

Скрипт build хийж, `tests/TestRunner.java`-г хөрвүүлж ажиллуулна (JUnit-гүй, цэвэр Java 17, ойролцоогоор 5--10 секунд). Төгсгөлд нь:

```
Нийт: 100, амжилттай: 100, алдаатай: 0
Хүснэгт: tests/results/test-results.md
```

`tests/results/test-results.md` файлд тест бүрийн **оролт, хүлээгдсэн утга, бодит утга, үр дүн** хүснэгтээр бичигдэнэ. Хүлээгдсэн утгыг тестийн кодонд урьдчилан бичсэн бөгөөд бодит утга нь програмыг ажиллуулж гарсан үр дүн юм. Бүх тест амжилттай бол exit code 0, үгүй бол 1.

Ангилал	Тоо	Шалгах зүйл
Номын жишээ	26	5 × 5 = 25 алгоритмын хослол, хуваалт ба хоёр хэсэг
Захын тохиолдол	17	n = 1, 2, 3, бүгд ижил, сөрөг, INT_MIN/MAX, эрэмбэтэй/урвуу, ⌈n/2⌉/⌊n/2⌋
Буруу оролт	18	Хоосон, тоо биш, бутархай, int-ээс гадуур, O(n^2)/trace/дээд хязгаар, буруу файл
Давхардал ба хэлбэр	6	4 хэлбэр × 25 хослол, 2000 санамсаргүй ажиллагаа
Том өгөгдөл	4	n = 1 000 000 (merge/quick), n = 20 000 (O(n^2))
Thread	5	3 тусдаа thread, параметр, TERMINATED, Merger-ийн дараалал, муж зөрчөөгүй
Event trace	7	Replay = бодит үр дүн, JSON формат, Parent-ийн event дараалал
Lockstep	5	WAITING, 1 permit = 1 үйлдэл, L/R ээлжлэх, дуусах, цуцлах
Командын мөр	12	Тусдаа JVM дээр гаралт ба exit code
Нийт	**100**	

Б6. Захын ба алдааны тохиолдлыг гараар шалгах

Exit code-ийг `echo $?` (Windows: `echo %errorlevel%`)-ээр харна.

Команд	Хүлээгдэх үр дүн	Exit
`./run.sh --file examples/book-example.txt`	`[2, 3, 4, 6, 7, 8, 12, 15, 18, 19]`, ЗӨВ	0
`./run.sh 42`	`[42]`, “баруун хэсэг хоосон” тэмдэглэл	0
`./run.sh 2 1`	`[1, 2]`	0
`./run.sh 3 1 2`	`[1, 2, 3]`, хуваалт 2 / 1	0
`./run.sh --file examples/duplicates.txt`	`[1, 1, 1, 2, 2, 3, 3, 3, 3, 5, 5, 5]`	0
`./run.sh --random 20 --pattern sorted --algo bubble`	ЗӨВ	0
`./run.sh --random 20 --pattern reversed --algo insertion`	ЗӨВ	0
`./run.sh --random 1000000 --seed 1 --algo quick`	ЗӨВ (секундээс бага)	0
`./run.sh`	Оролтын өгөгдөл өгөөгүй байна	2
`./run.sh --input ""`	Оролт хоосон байна	2
`./run.sh --input "5,3,abc"`	3-р утга 'abc' нь бүхэл тоо биш.	2
`./run.sh --input "1,2.5"`	Бутархай тооны алдаа	2
`./run.sh --input "1,3000000000"`	int мужийн алдаа	2

Команд	Хүлээгдэх үр дүн	Exit
`./run.sh --random 20001 --algo insertion`	O(n^2) хязгаарын алдаа	2
`./run.sh --random 20001 --algo insertion --force`	Анхааруулга, ЗӨВ	0
`./run.sh --random 5 --min 9 --max 1`	min (9) нь max (1)-аас их байж болохгүй.	2
`./run.sh --input "1,2" --algo foo`	'foo' гэсэн алгоритм байхгүй	2

Б7. Event trace (JSON)

```
./run.sh --file examples/book-example.txt --algo-left insertion --algo-right quick
↪  --trace book.trace.json
```

Бүх алхмыг `book.trace.json` файлд `ii-4-6-trace/1` форматаар хадгалж, terminal дээр event-ийн тоог төрлөөр болон thread-ээр хэвлэнэ. `*.trace.json` файлууд `.gitignore`-д орсон.

Файлын бүтэц:

- `format`, `n`, `mid`, `algorithms`, `input`, `result`, `sorted`;
- `threads`: thread бүрийн нэр, tid, муж, үйлдлийн тоо, эхэлсэн/дууссан хугацаа;
- `timing`: `tracedTotalNs` (trace-тэй), `untracedTotalNs` (trace-гүй, 2 дулаацуулалтын дараах 3 ажиллалтын медиан);
- `eventCounts`, `eventCount`, `truncated`;
- `events`: `{seq, t, th, type, a, b, c, note}` объектуудын жагсаалт.

`type`	thread	Утга
`RUN_START, START, JOIN, RUN_END`	Parent	Run эхлэх, `start()`, `join()`, дуусах
`T_START, T_DONE`	Sorter	Хэсгээ эрэмбэлж эхэлсэн / дууссан
`CMP`	Sorter	`list[a]`-г `list[b]`-тэй харьцуулав, `c` = тэмдэг
`CMPV`	Sorter	`list[a]`-г утга `b` (pivot)-тэй харьцуулав
`CMPA`	Sorter	Merge sort-ын туслах буфер дахь харьцуулалт
`SWP`	Sorter	`list[a]` ↔ `list[b]`
`SET`	Sorter	`list[a] = b`
`M_START, M_DONE`	Merger	Нэгтгэл эхэлсэн / дууссан
`MCMP`	Merger	`list[a]` (зүүн) ба `list[b]` (баруун)-ийг харьцуулав
`MW`	Merger	`result[a] = list[b]`, `c` = бичсэн утга

Trace-ийн event-үүдийг эх жагсаалт дээр дахин хэрэглэвэл (replay) бодит `result[]` яг гарч ирдэг. Үүнийг автомат тест шалгадаг.

Б8. Мэдэгдэж буй асуудал

1. **Windows скрипт туршаагүй.** `build.bat`, `run.bat`, `test.bat`-ийг бодит Windows машин дээр ажиллуулж үзээгүй. Асуудал гарвал Command Prompt дээр гараар build хийж болно (хавтасны замд зай байхгүй үед):

   ```
   dir /s /b src\main\java\*.java > sources.txt
   javac --release 17 -encoding UTF-8 -d out @sources.txt
   xcopy /e /i /y src\main\resources out
   java -Dfile.encoding=UTF-8 -cp out sorting.Main --web
   ```

2. **Хугацааны хэмжилт шуугиантай.** n бага үед хугацаа миллисекундээс бага бөгөөд JIT, thread үүсгэх зардал, OS-ийн scheduler-ээс хамаарч ажиллах бүрд өөр гарна. Мөн n бага үед thread үүсгэх зардал эрэмбэлэх ажлаас их байдаг тул Sorter-L Sorter-R эхлэхээс өмнө дуусчихсан байж болно. Энэ нь алдаа биш. Тиймээс CLI ба trace нь дулаацуулсны дараах медиан хугацааг тусад нь хэвлэдэг.
3. **Trace-тэй хугацаа удаан.** Event бүрийг бүртгэх зардлаас болж trace-тэй ажиллалт удаан. Хурдыг харьцуулахдаа “trace-гүй” хугацааг ашиглана.
4. **Lockstep-д хойш алхах боломжгүй.** Бодит thread-ийг буцааж болохгүй. Хугацааны тэмдэглэгээ нь хэрэглэгчийн товч дарах хоорондын хугацааг агуулна.
5. **Lockstep-ийн эхний permit.** Аль sorter нь gate-д түрүүлж хүрэх нь OS-ийн scheduler-ээс хамаарна. Түүнээс хойш fair semaphore хоёр sorter-ийг ээлжлүүлнэ.
6. **Web сервер зөвхөн локал.** `127.0.0.1` дээр сонсдог тул өөр компьютерээс нээгдэхгүй (зориуд ингэсэн).

Б9. Файлын бүтэц

```
F.CSM302-TeamProject-2-4-6/
|-- README.md
|-- build.sh, build.bat          хөрвүүлэх
|-- run.sh, run.bat              ажиллуулах
|-- test.sh, test.bat            автомат тест
|-- src/main/java/sorting/
|   |-- Main.java, Config.java   CLI ролт, сонголт
|   |-- algo/                    5 эрэмбэлэх алгоритм
|   |-- core/                    SortingThread, MergingThread, MultithreadedSorter,
↪  StepGate, Ops ...
|   |-- input/                   InputParser, Limits, DataGenerator
|   |-- trace/                   TraceRecorder, TraceEvent, TraceJson
|   `-- web/                     WebServer, LiveSession
|-- src/main/resources/web/      index.html, app.js, style.css
|-- tests/TestRunner.java        100 автомат тест
|-- tests/results/               test-results.md
|-- examples/                    тестийн өгөгдөл
|-- docs/                        Manual.md, manual/manual.tex, images/
`-- report/                      LaTeX тайлан
```

Энэ гарын авлагын нооргийг Claude (AI) ашиглан бэлтгэсэн. Багийн гишүүд уншиж, командуудыг ажиллуулж шалгасан.